

Claude Code Prompting Guide

Adapted from the "Role Prompting — Reference List" and "Chain of Thought (CoT) — Variations" references, realigned for **Claude Code** instead of Copilot.

Two things are different in Claude Code and worth knowing before you use these patterns:

- **Roles and reasoning instructions are still just plain-text prompt content** — Claude Code doesn't have a special "role" API, so `Acting as a <role>, ...` and CoT trigger phrases work exactly as prose, typed directly into the chat or embedded in a `CLAUDE.md` / skill file.
- **Claude Code has native mechanics that overlap with several CoT variations** — plan mode, subagents, todo lists, and skills. Where a pattern maps onto one of these, it's called out so you use the built-in mechanism instead of hand-rolling it in a prompt.

Part 1 — Role Prompting

Use as: `Acting as a <role>, ...`

The role shifts Claude's lens — what it prioritizes, what it flags, what vocabulary it uses. The underlying model doesn't change; only emphasis does.

Three ways to apply a role in Claude Code

1. **Inline, one-off** — prefix your message: `Acting as a senior SRE, review this deployment script for single points of failure.`
2. **Persistent, project-wide** — put the role in `CLAUDE.md` (e.g. "When reviewing SQL in this repo, act as a DBA focused on production safety") so it applies to every session in this project without repeating it.
3. **Delegated, isolated** — launch a subagent with the `Agent` tool and give it the role in its prompt. Useful when you want the persona's review to happen in its own context window without polluting the main conversation (e.g. spin up a fresh agent as "a penetration tester" to audit an endpoint while you keep working on something else).

Engineering Roles

Role	What it shifts
Senior TypeScript / Java / Python / Go engineer	Language idioms, best practices for that ecosystem
Backend engineer	API design, data access patterns, server-side concerns
Frontend engineer	Component design, accessibility, bundle size awareness
Full-stack developer	End-to-end tradeoffs, API contract ownership
Systems programmer	Memory management, performance, low-level concerns
Embedded / firmware engineer	Resource constraints, determinism, no dynamic allocation
DevOps / Platform engineer	Infra-as-code, CI/CD, observability, deployment safety
Site Reliability Engineer (SRE)	Reliability, SLOs, error budgets, graceful degradation
Data engineer	Pipeline design, schema evolution, idempotency
ML engineer	Data preprocessing, model integration, inference efficiency

Architecture Roles

Role	What it shifts
Software architect	Design patterns, SOLID, long-term maintainability
Solution architect	System-level tradeoffs, component boundaries, vendor choice
Enterprise architect	Governance, standards compliance, org-level consistency
API designer	REST/GraphQL conventions, versioning, consumer-first thinking
Database architect	Normalisation, indexing strategy, query performance
Cloud architect (AWS / Azure / GCP)	Managed services, cost optimisation, cloud-native patterns
Microservices architect	Service boundaries, coupling, distributed systems tradeoffs
Event-driven architect	Messaging patterns, eventual consistency, idempotency

Security Roles

Role	What it shifts
Application security engineer	OWASP Top 10, input validation, secure defaults
Penetration tester	Attacker mindset — "how would I exploit this?"
Security auditor	Compliance evidence, audit trail, policy adherence
Cryptographer	Algorithm selection, key management, random number generation
Identity / IAM engineer	Auth flows, token handling, privilege escalation risks
Data privacy engineer	PII exposure, GDPR / DPDP compliance, data minimisation

For security work, also consider the built-in `/security-review` skill instead of (or alongside) a role prompt — it runs a structured review of pending changes rather than relying on persona framing alone.

Quality & Testing Roles

Role	What it shifts
Senior code reviewer	Readability, naming, abstraction leaks, PR comments
QA / Test engineer	Edge cases, boundary conditions, test coverage gaps
Performance engineer	Algorithmic complexity, memory allocation, profiling approach
Accessibility expert	WCAG compliance, screen reader compatibility, keyboard navigation
Localisation engineer	i18n patterns, string externalisation, locale-aware formatting

For code review specifically, the `/code-review` skill (with effort levels `low` → `ultra`) is the native equivalent of "acting as a senior code reviewer" — use it when you want structured, ranked findings rather than freeform commentary.

Data & Database Roles

Role	What it shifts
Database administrator (DBA)	Index strategy, locking, connection pooling, vacuum/maintenance
SQL expert	Query optimisation, execution plans, join strategy
Data modeller	Entity relationships, normal forms, schema evolution
Elasticsearch engineer	Mapping design, query DSL, relevance tuning
Redis architect	Data structure selection, eviction policy, persistence tradeoffs

Documentation & Communication Roles

Role	What it shifts
Technical writer	Clarity, audience-appropriate language, structure
API documentation specialist	OpenAPI conventions, examples, error descriptions
Developer advocate	Tutorial-style explanation, onboarding experience
Rubber duck debugger	Forces step-by-step reasoning aloud — great for debugging prompts

Domain Expert Roles (Industry)

Role	What it shifts
Financial systems engineer	Precision arithmetic, audit requirements, regulatory constraints
Payments engineer	PCI-DSS awareness, idempotency, double-spend prevention
Healthcare data engineer	HIPAA / HL7 / FHIR, PHI handling, consent management
E-commerce engineer	Inventory concurrency, cart consistency, tax calculation edge cases
Compliance engineer	Regulatory mapping, evidence collection, policy implementation

Teaching & Explanation Roles

Role	What it shifts
Senior engineer mentoring a junior	Explains the why, not just the what. Adds comments.
Computer science professor	Theory-first, complexity analysis, formal definitions
Code explainer	Line-by-line walkthrough, plain English, no jargon
Interviewer	Asks clarifying questions before answering — surfaces assumptions

Quick Examples

Acting as a senior SRE, review this deployment script for single points of failure.

Acting as a penetration tester, identify how this login endpoint could be abused.

Acting as a DBA, rewrite this query to avoid a full table scan.

Acting as a technical writer, rewrite this README for a developer new to the codebase.

Acting as a payments engineer, review this refund flow for idempotency issues.

Acting as a senior code reviewer, give me the 3 most important comments you would leave on this pull request.

Acting as a computer science professor, explain the time complexity of this algorithm and suggest a more efficient approach.

Tips for Role Prompting

- **Be specific.** "Senior TypeScript engineer" produces better output than "software engineer."
- **Combine roles when needed:** "Acting as a senior Java engineer with a focus on Spring Security..."
- **The role changes emphasis, not facts.** Claude still uses the same model — the role shifts what it prioritises and flags.
- **Stack with constraints:** Role prompting + explicit constraints = very precise output.

Acting as a DBA reviewing for production safety.

CONSTRAINTS: flag anything that could cause a table lock, full scan, or N+1 query.

- **Stack with subagents:** For an independent second opinion, delegate the roled prompt to a fresh subagent via the `Agent` tool rather than asking the same conversation to switch hats — it won't be anchored by the reasoning already in your context.

Part 2 — Chain of Thought (CoT) Variations

CoT tells Claude to *reason before it answers*. The trigger phrase is the variation. Each one changes how the reasoning unfolds. Several map directly onto Claude Code's built-in mechanics — use those instead of reinventing them in a prompt where noted.

1. Basic CoT — List then Build

The simplest form. Force enumeration before generation.

Before writing any code, list every edge case that
`parseAmount(value: string): number` must handle.
Then write the function addressing each one.

2. Step-by-Step CoT

The minimal trigger. Works on almost any problem.

Think step by step. How would you design a rate limiter
that supports multiple algorithms (fixed window, sliding window,

token bucket) without changing the calling code?

3. Plan-then-Execute

Produce a plan first. Execute only after the plan is approved.

Claude Code native equivalent: *this is what **plan mode** is for. Ask Claude to enter plan mode (or it will propose entering it for non-trivial work) — it researches and proposes a plan without editing files, you approve or redirect it, and only then does it execute. Prefer this over a manual "don't write code yet" prompt when you want a reviewable, structured plan rather than prose.*

First, write a numbered implementation plan for adding soft-delete to the User entity without breaking existing queries. Do not write any code yet.

(Review the plan, then follow up:)

The plan looks good. Now execute step 3 only.

4. Self-Critique CoT

Write it. Critique it. Improve it. Three passes in one prompt.

Write a function that merges two sorted arrays.
Then critique your own implementation – identify any edge cases missed, any inefficiencies, or any readability issues.
Then write an improved version based on your critique.

For critiquing existing code rather than something just written, `/code-review` (optionally with `--fix` to apply findings) gives a more structured version of this pattern.

5. Failure Mode Analysis

Enumerate ways it breaks in production before writing anything.

Before implementing this caching layer, list every way it could fail or cause incorrect behaviour in production:

- race conditions
- cache invalidation problems
- memory leaks
- stale data scenarios
- failure when the cache is unavailable

Then write the implementation addressing each failure mode.

6. Adversarial CoT (Red Team)

Argue against your own approach first.

I want to use a single global event bus for inter-service communication in our monolith.

First, give me the strongest 3 arguments AGAINST this approach.

Then, given those arguments, recommend whether to proceed and what mitigations to apply if we do.

7. Hypothesis Testing

Form a hypothesis. Test it against the code.

The bug is that orders occasionally get a 'duplicate key' error on insert. My hypothesis is that the UUID generation is not truly unique under concurrent load.

What evidence in the following code would support or refute this hypothesis? Test it systematically.

[paste code]

8. Reverse CoT (Work Backwards)

Start from the output, reason backwards to the cause.

This function is producing incorrect totals for orders that contain discount codes. Here is the wrong output:

Input: [{ price: 100, qty: 2 }, { price: 50, qty: 1 }], discount: 0.1
Expected total: 225
Actual total: 240

Work backwards from the incorrect output to identify where in the calculation the error occurs, then fix it.

9. Analogical CoT

Use an analogy to shape the reasoning.

Think of a database connection pool like a car park with a fixed number of spaces. Using that mental model, explain why the following configuration will cause timeouts under load, and what the correct settings should be:

max: 5
idleTimeoutMillis: 30000
connectionTimeoutMillis: 2000

10. Least-to-Most CoT

Solve the simplest sub-problem first, build up to the full problem.

Claude Code native equivalent: for a multi-step build like this, ask Claude to track the steps as a todo list — it keeps each stage visible and checked off as it completes, which mirrors this pattern's incremental structure.

Solve this in increasing complexity:

Step 1: Write a function that checks if a single password

meets length requirements (min 8 chars).

Step 2: Extend it to also check for at least one uppercase and one digit.

Step 3: Extend it to return structured validation errors (not just true/false) so the UI can show specific messages.

Step 4: Make it configurable – accept a PasswordPolicy object so rules can change without modifying the function.

11. Contrastive CoT

Show a bad reasoning path alongside the good one. Claude learns from the contrast.

Here are two approaches to handling the N+1 query problem.

BAD reasoning:

"Just add `.include()` everywhere and it'll be fine."

Result: loads 10,000 records when you need 10.

GOOD reasoning:

"Identify the access pattern first. Load only what the caller will use. Use a DataLoader for batching if called in a loop."

Apply the GOOD reasoning pattern to fix the following repository method: [paste method]

12. Socratic CoT

Claude asks clarifying questions before answering.

I want to add full-text search to our product catalogue.
Before recommending an approach, ask me the 5 most important clarifying questions whose answers would change your recommendation.
Then, after I answer, give your recommendation.

13. Complexity-First CoT

Reason about the optimal solution before implementing.

Before writing any code:

1. What is the optimal time complexity for finding the k most frequent elements in an array?
2. What data structure achieves that complexity?
3. What are the tradeoffs vs a simpler $O(n \log n)$ approach?

Then implement the optimal solution with that reasoning as comments.

14. Test-First CoT

Reason about the tests before writing the implementation.

Before implementing `calculateShippingCost(order: Order): Money``:

1. List the test cases that a correct implementation must pass, grouped by: happy path / boundary conditions / error cases.
2. Then write the implementation that would pass all of them.

15. Security-First CoT

Reason about threats before implementing.

For a full pass rather than an inline prompt, the `/security-review` skill runs this pattern as a structured review of pending changes end-to-end.

Before implementing the file upload endpoint:

1. List every security risk this feature introduces.
2. For each risk, state the mitigation.
3. Then write the implementation with those mitigations built in from the start.

Quick Comparison

Variation	Best for
Basic CoT	Edge case discovery
Step-by-step	General reasoning on any complex task
Plan-then-execute	Large changes where you want to review the approach first — use plan mode
Self-critique	Getting a better answer than the first pass
Failure mode analysis	Anything that touches production data or state
Adversarial	Architecture decisions — stress-test your own idea
Hypothesis testing	Debugging — structured root cause analysis
Reverse CoT	Bug fixing from observed wrong output
Analogical	Explaining or reasoning about complex abstractions
Least-to-most	Teaching, or building complex logic incrementally — track with a todo list
Contrastive	Correcting a pattern Claude keeps repeating badly
Socratic	When you're not sure what information Claude needs
Complexity-first	Algorithm design, performance-sensitive code
Test-first	TDD workflow, acceptance criteria from requirements
Security-first	Auth, file handling, external input, payment flows — see <code>/security-review</code>

Combining Variations

Variations stack. The most powerful patterns combine two:

```
/* Plan-then-Execute + Failure Mode Analysis */
```

```
Plan the implementation of our webhook signature verification.  
Before the plan, list every way signature verification can be  
bypassed or fail silently. Address each in the plan.  
Then execute step 1 of the plan only.
```

```
/* Self-Critique + Security-First */
```

```
Write the password reset flow.  
Then critique it as a penetration tester looking for  
account takeover vulnerabilities.  
Then produce a final version with all vulnerabilities addressed.
```

Combining Part 1 and Part 2

Role prompting and CoT variations compose freely — a role sets *whose* lens is applied, a CoT variation sets *how* the reasoning unfolds:

```
Acting as a payments engineer, use Failure Mode Analysis before  
implementing the refund endpoint: list every way a refund could be  
double-processed or lost, then implement addressing each one.
```